

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7

дисциплина: *Архитектура компьютера*

Студент: Киракосян Арман Тигранович

Группа: НПИбд-01-25

МОСКВА

2025г.

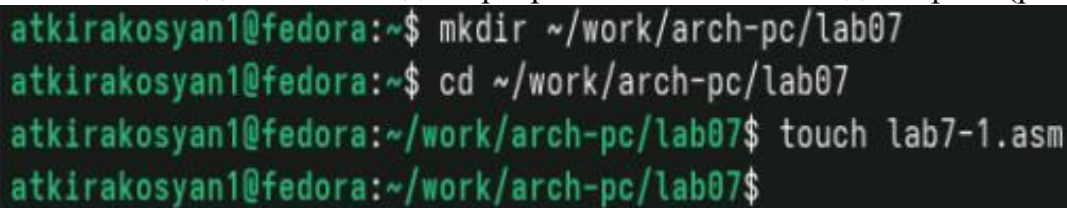
1. Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

2. Выполнение лабораторной работы

2.1 Реализация переходов в NASM

Создаем каталог для программ ЛБ7 и в нем создаем файл (рис.2.1)

A terminal window with a black background and green text. It shows four lines of commands and their execution. The first line creates a directory, the second changes to it, the third creates a file, and the fourth shows the prompt after the file is created.

```
atkirakosyan1@fedora:~$ mkdir ~/work/arch-pc/lab07
atkirakosyan1@fedora:~$ cd ~/work/arch-pc/lab07
atkirakosyan1@fedora:~/work/arch-pc/lab07$ touch lab7-1.asm
atkirakosyan1@fedora:~/work/arch-pc/lab07$
```

Рис. 2.1 Создаем каталог с помощью команды mkdir и файл с помощью команды touch

Заполняем созданный файл lab7-1 в соответствии с листингом 7.1 (рис. 2.2)

```
lab7-1.asm      [----] 41 L:[ 1+19 20/ 20] *(649 / 649b) <EOF>      [*][X]
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
_end:
call quit ; вызов подпрограммы завершения
```

Рис. 2.2 Заполняем файл

Создаем исполняемый файл и запускаем его (рис. 2.3)

```
atkirakosyan1@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 3
atkirakosyan1@fedora:~/work/arch-pc/lab07$
```

Рис. 2.3 Запускаем файл и смотрим на его работу

Снова открываем файл для редактирования и изменяем его в соответствии с листингом 7.2 (рис. 2.4)

```
lab7-1.asm [-M--] 11 L:[ 1+21 22/ 22] *(613 / 670b) 0032 0x020 [*][X]
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
jmp _label1
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
_end:
call quit ; вызов подпрограммы завершения
```

Рис. 2.4 Изменяем файл

Создаем исполняемый файл и запускаем его (рис. 2.5).

```
atkirakosyan1@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 1
atkirakosyan1@fedora:~/work/arch-pc/lab07$
```

Рис. 2.5 Запускаем файл и смотрим на его работу

Снова открываем файл для редактирования и изменяем его чтобы произошел данный вывод (рис. 2.6)

```
lab7-1.asm [-M--] 11 L:[ 1+16 17/ 23] *(487 / 682b) 0010 0x00A [*][X]
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label3
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
jmp _label1
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
jmp _label2
_end:
call quit ; вызов подпрограммы завершения
```

Рис. 2.6 Редактируем файл

Создаем исполняемый файл и запускаем его (рис. 2.7).

```
atkirakosyan1@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1
atkirakosyan1@fedora:~/work/arch-pc/lab07$
```

Рис. 2.7 Проверяем, сошелся ли наш вывод с данным в условии выводом

Создаем новый файл (рис. 2.8)

```
atkirakosyan1@fedora:~/work/arch-pc/lab07$ touch lab7-2.asm
atkirakosyan1@fedora:~/work/arch-pc/lab07$
```

Рис. 2.8 Создаем файл командой touch

Открываем файл в Midnight Commander и заполняем его в соответствии с листингом

7.3 (рис. 2.9)

```
lab7-2.asm [-M--] 10 L: [ 1+18 19/ 49] *(352 /1743b) 0010 0x00A
#include 'in_out.asm'
section .data
msg1 db 'Введите B: ',0h
msg2 db 'Наибольшее число: ',0h
; dd '20'
; dd '50'
section .bss
max resb 10
B resb 10
section .text
global _start
_start:
; ----- Вывод сообщения 'Введите B: '
mov eax,msg1
call sprint
; ----- Ввод 'B'
mov ecx,B
mov edx,10
call sread
; ----- Преобразование 'B' из символа в число
mov eax,B
call atoi ; Вызов подпрограммы перевода символа в число
mov [B],eax ; запись преобразованного числа в 'B'
; ----- Записываем 'A' в переменную 'max'
mov ecx,[A] ; 'ecx = A'
mov [max],ecx ; 'max = A'
; ----- Сравниваем 'A' и 'C' (как символы)
cmp ecx,[C] ; Сравниваем 'A' и 'C'
jg check_B ; если 'A>C', то переход на метку 'check_B',
mov ecx,[C] ; иначе 'ecx = C'
mov [max],ecx ; 'max = C'
; ----- Преобразование 'max(A,C)' из символа в число
check_B:
mov eax,max
call atoi ; Вызов подпрограммы перевода символа в число
mov [max],eax ; запись преобразованного числа в 'max'
; ----- Сравниваем 'max(A,C)' и 'B' (как числа)
mov ecx,[max]
cmp ecx,[B] ; Сравниваем 'max(A,C)' и 'B'
jg fin ; если 'max(A,C)>B', то переход на 'fin',
mov ecx,[B] ; иначе 'ecx = B'
mov [max],ecx
; ----- Вывод результата
fin:
mov eax,msg2
call sprint ; Вывод сообщения 'Наибольшее число: '
mov eax,[max]
call iprintLF ; Вывод 'max(A,B,C)'
call quit ; Выход
```

Рис. 2.9 Заполняем файл

Создаем исполняемый файл и проверяем его работу, вводя разные значения B (рис. 2.10).


```
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o
atkirakosyan1@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 30
Наибольшее число: 50
atkirakosyan1@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 50
Наибольшее число: 50
atkirakosyan1@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 100
Наибольшее число: 100
atkirakosyan1@fedora:~/work/arch-pc/lab07$
```

Рис. 2.10 Смотрим на работу программ

2.2 Изучение структуры файла листинга

Создаем файл листинга для программы lab7-2.asm (рис. 2.11)

```
atkirakosyan1@fedora:~/work/arch-pc/lab07$ nasm -f elf -l lab7-2.lst lab7-2.asm
atkirakosyan1@fedora:~/work/arch-pc/lab07$
```

Рис.2.11 Создаем файл листинга

Открываем файл листинга с помощью команды mcedit и изучаем его (рис. 2.12)

```
lab7-2.lst [----] 22 L:[ 11+30 41/225] *(2492/14458b) 0032 0x020 [*][x]
10 00000008 40 <1> inc eax.....
11 00000009 EBF8 <1> jmp nextchar.....
12 <1>.....
13 <1> finished:
14 0000000B 29D8 <1> sub eax, ebx
15 0000000D 5B <1> pop ebx.....
16 0000000E C3 <1> ret.....
17 <1>.
18 <1>.
19 <1> ;----- sprint -----
20 <1> ; Функция печати сообщения
21 <1> ; входные данные: mov eax,<message>
22 <1> sprint:
23 0000000F 52 <1> push edx
24 00000010 51 <1> push ecx
25 00000011 53 <1> push ebx
26 00000012 50 <1> push eax
27 00000013 E8E8FFFFFF <1> call slen
28 <1>.....
29 00000018 89C2 <1> mov edx, eax
30 0000001A 58 <1> pop eax
31 <1>.....
32 0000001B 89C1 <1> mov ecx, eax
33 0000001D BB01000000 <1> mov ebx, 1
34 00000022 B804000000 <1> mov eax, 4
35 00000027 CD80 <1> int 80h
36 <1>.
37 00000029 5B <1> pop ebx
38 0000002A 59 <1> pop ecx
39 0000002B 5A <1> pop edx
40 0000002C C3 <1> ret
1 Помощь 2 Сохран 3 Блок 4 Замена 5 Копия 6 Пере-твить 7 Поиск 8 Удалить 9 МенюМС 10 Выход
```

Рис. 2.12 Изучаем файл

Строка 33: 0000001D-адрес в сегменте кода, BB01000000-машинный код, mov ebx, 1-

присвоение переменной есх значения 1.

Строка 34: 00000022-адрес в сегменте кода, B804000000-машинный код, mov еах, 4-
присвоение переменной еах значения 4.

Строка 35 00000027-адрес в сегменте кода, CD80-машинный код, int 80h-вызов ядра.

Открываем файл и удаляем один операндум (рис.2.13).

```
lab7-2.asm      [-M--]  7 L:[ 1+17 18/ 49] *(338 /
%include 'in_out.asm'
section .data
msg1 db 'Введите B: ',0h
msg2 db "Наибольшее число: ",0h
A dd '20'
C dd '50'
section .bss
max resb 10
B resb 10
section .text
global _start
_start:
; ----- Вывод сообщения 'Введите B: '
mov eax,msg1
call sprint
; ----- Ввод 'B'
mov ecx,B
mov edx
call sread
; ----- Преобразование 'B' из символа в число
mov eax,B
call atoi ; Вызов подпрограммы перевода символа в число
mov [B],eax ; запись преобразованного числа в 'B'
; ----- Записываем 'A' в переменную 'max'
mov ecx,[A] ; 'ecx = A'
mov [max],ecx ; 'max = A'
; ----- Сравниваем 'A' и 'C' (как символы)
cmp ecx,[C] ; Сравниваем 'A' и 'C'
jg check_B ; если 'A>C', то переход на метку 'check_B',
mov ecx,[C] ; иначе 'ecx = C'
mov [max],ecx ; 'max = C'
```

Рис. 2.13 Удаляем операндум из файла

Транслируем с получением файла листинга (рис. 2.14).

```
atkirakosyan1@fedora:~/work/arch-pc/lab07$ nasm -f elf -l lab7-2.lst lab7-2.asm
lab7-2.asm:18: error: invalid combination of opcode and operands
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ls
in_out.asm  lab7-1  lab7-1.asm  lab7-1.o  lab7-2  lab7-2.asm  lab7-2.lst  lac7-1
atkirakosyan1@fedora:~/work/arch-pc/lab07$
```

Рис. 2.14 Транслируем файл

При трансляции файла выдается ошибка, но создаются исполнительный файл lab7-2 и lab7-2.lst

Снова открываем файл листинга и изучаем его (рис. 2.15).

```
lab7-2.lst [-----] 22 L: [101+14 115/226] * (7180/14546b) 0032 8x020
100 <1>
101 00000001 5E <1> pop esi...
102 00000002 5A <1> pop edx...
103 00000003 59 <1> pop ecx...
104 00000004 58 <1> pop eax...
105 00000005 C3 <1> ret
106 <1>
107 <1>
108 <1> ;----- iprintf -----
109 <1> ; Функция вывода на экран чисел в формате ASCII
110 <1> ; входные данные: mov eax,<int>
111 <1> iprintf:
112 00000006 E8C9FFFFFF <1> call iprint.....
113 <1>
114 00000008 50 <1> push eax.....
115 0000000C B80A000000 <1> mov eax, 0Ah.....
116 00000001 50 <1> push eax.....
117 00000002 80EB <1> mov eax, esp.....
118 00000004 E876FFFFFF <1> call sprintf.....
119 00000009 58 <1> pop eax.....
120 0000000A 58 <1> pop eax.....
121 0000000B C3 <1> ret
122 <1>
123 <1> ;----- atoi -----
124 <1> ; Функция преобразования ascii-код символа в целое число
125 <1> ; входные данные: mov eax,<int>
126 <1> atoi:
127 0000000C 53 <1> push ebx.....
128 0000000D 51 <1> push ecx.....
129 0000000E 52 <1> push edx.....
130 0000000F 56 <1> push esi.....
131 000000A0 89C6 <1> mov esi, eax.....
132 000000A2 B800000000 <1> mov eax, 0.....
133 000000A7 B900000000 <1> mov ecx, 0.....
134 <1>..
135 <1> .multiplyLoop:
136 000000AC 310B <1> xor ebx, ebx.....
137 000000AE 8A1C0E <1> mov bl, [esi+ecx]
138 000000B1 80FB30 <1> cmp bl, 40.
139 000000B4 7C14 <1> jl .finished.
140 000000B6 80FB39 <1> cmp bl, 57..
141 000000B9 7FBF <1> jg .finished.
142 <1>..
143 000000BB 80EB30 <1> sub bl, 40.
144 000000BE 010B <1> add eax, ebx
145 000000C0 B80A000000 <1> mov ebx, 10..
146 000000C5 F7E3 <1> mul ebx..
147 000000C7 41 <1> inc ecx...
148 000000C8 EB02 <1> jmp .multiplyLoop..
149 <1>..
1 Помощь 2 Сохран 3 Блок 4 Замена 5 Поиск 6 Переместить 7 Поиск 8 Удалить 9 Меню MS 10 Выход
```

Рис. 2.15 Анализируем файл с ошибкой

Создается ТОЛЬКО файл листинга `lab7-2.lst`. Файл объектного кода lab7-2.o НЕ создается, потому что ассемблер обнаружил ошибку и остановил компиляцию. При наличии синтаксических ошибок в исходном коде: - Объектный файл (.o) не создается - компиляция прерывается - Листинг создается, но содержит сообщения об ошибках - это помогает локализовать проблему - Ассемблер указывает точное место и характер ошибки - в данном случае "ожидалась инструкция" из-за неполной команды

3. Задание для самостоятельной работы

ВАРИАНТ 17

1. Напишите программу нахождения наименьшей из 3 целочисленных переменных a и c . Значения переменных выбрать из табл. 7.5 в соответствии с вариантом, полученным при выполнении лабораторной работы № 6. Создайте исполняемый файл и проверьте его работу.

Создаем новый файл (рис. 3.1)

```
atkirakosyan1@fedora:~/work/arch-pc/lab07$ touch lab7-3.asm
atkirakosyan1@fedora:~/work/arch-pc/lab07$
```

Рис. 3.1 Создаем файл командой touch и редактируем в Midnight Commander

Открываем его и пишем программу, которая выберет наименьшее число из трех (2 числа уже в программе, третье вводится с консоли)(рис. 3.2)

```
lab7-3.asm [----] 12 L: [ 1+ 8 1/ 41] *(12 / 763b) 8895 8x85F
%include 'inout.asm'
section .data
    msg1 db 'Введите B: ',0h
    msg2 db 'Наименьшее число: ',0h
    A dd '83'
    C dd '38'
section .bss
    min resb 10
    B resb 10
section .text
    global _start
_start:
    mov eax,msg1
    call sprint
    mov ecx,B
    mov edx,10
    call sread
    mov eax,B
    call atoi
    mov [B],eax
    mov ecx,[A]
    mov [min],ecx
    cmp ecx,[C]
    jl check_B
    mov ecx,[C]
    mov [min],ecx
check_B:
    mov eax,min
    call atoi
    mov [min],eax
    mov ecx,[min]
    cmp ecx,[B]
    jl fin
    mov ecx,[B]
    mov [min],ecx
fin:
    mov eax,msg2
    call sprint
    mov eax,[min]
    call iprintLF
    call quit
```

Рис. 3.2 Прописываем программу

Транслируем файл и смотрим на работу программы (рис. 3.3)

```
atkirakosyan1@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-3.asm
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-3 lab7-3.o
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ./lab7-3
Введите В: 73
Наименьшее число: 30
atkirakosyan1@fedora:~/work/arch-pc/lab07$
```

Рис. 3.3 Смотрим на работу выполненного файла (всё верно)

2. Напишите программу, которая для введенных с клавиатуры значений x и a вычисляет значение заданной функции $f(x)$ и выводит результат вычислений. Вид функции $f(x)$ выбрать из таблицы 7.6 вариантов заданий в соответствии с вариантом, полученным при выполнении лабораторной работы № 6. Создайте исполняемый файл и проверьте его работу для значений x и a из 7.6.

Создаем новый файл (рис. 3.4).

```
atkirakosyan1@fedora:~/work/arch-pc/lab07$ touch lab7-4.asm  
atkirakosyan1@fedora:~/work/arch-pc/lab07$
```

Рис. 3.4 Создаем файл командой touch

Открываем его и пишем программу, которая решит систему уравнений при данных, введенных в консоль (рис. 3.5)

```
lab7-4.asm 713 [0-1-33 34/ 45] * (578 / 7485) 8532 8x020  
*include "in_out.asm"  
SECTION .data  
    msg1: DB "Введите x: ",0  
    msg2: DB "Введите a: ",0  
    otv: DB "F(x) = ",0  
SECTION .bss  
    x: RESB 80  
    a: RESB 80  
    res: RESB 80  
SECTION .text  
    GLOBAL _start  
_start:  
    mov eax, msg1  
    call sprintf  
    mov ecx, x  
    mov edx, 80  
    call sread  
    mov eax, x  
    call atoi  
    mov [x], eax  
    mov eax, msg2  
    call sprintf  
    mov ecx, a  
    mov edx, 80  
    call sread  
    mov eax, a  
    call atoi  
    mov [a], eax  
    cmp ecx, 1  
    jne calc_square  
    mov ecx, 10  
    add ecx, [x]  
    mov [res], ecx  
    jmp fin  
calc_square:  
    mov ecx, [a]  
    imul ecx, ecx  
    mov [res], ecx  
fin:  
    mov eax, otv  
    call sprintf  
    mov ecx, [res]  
    call iprintf  
    call quit
```

Рис. 3.5 Прописываем программу

Транслируем файл и проверяем его работу при $x = 1$ и $a = 2$ (рис. 3.6).

```
atkirakosyan1@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-4.asm  
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-4 lab7-4.o  
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ./lab7-4  
Введите x: 1  
Введите a: 2  
F(x) = 4
```

Рис 3.6 Проверяем работу программы (работает верно)

Транслируем файл и проверяем его работу при $x = 2$ и $a = 1$ (рис 3.7)

```
atkirakosyan1@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-4.asm
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-4 lab7-4.o
atkirakosyan1@fedora:~/work/arch-pc/lab07$ ./lab7-4
Введите x: 2
Введите a: 1
F(x) = 12
```

Рис. 3.7 Проверяем работу программы (работает верно)

4 Выводы

Мы научились решать программы с использованием циклов и обработкой аргументов командной строки.